Agent plugins in VS Code

Agent plugins package related customizations, such as skills, agents, and tool integrations, so you can install them together. For example, a testing plugin can include a test-running skill, review agent, and reporting integration that are intended to work together.

This article shows you how to discover, install, and manage plugins with the Agent Customizations editor, and how to troubleshoot common issues. To create or package a plugin, see the agent plugin authoring reference.

If you only need a project rule or a single skill, start with an individual customization. Choose a bundle when its components support a workflow you want to adopt together.

> **Note**
>
> Enable or disable support for agent plugins with the `chat.plugins.enabled` setting.

What plugins provide

Plugins can provide the following customizations. The selected harness and plugin format determine which components are available.

Expand table

Capability	Purpose
Skills	Instructions, scripts, and resources for specialized workflows
MCP servers	Tools for interacting with external systems
Canvases	Interactive visual surfaces shared by you and the agent in supported Copilot sessions
Custom agents	Specialized roles and tool configurations
Hooks	Commands that run at agent lifecycle points
Automation templates	Starting prompts and schedules to review before creating an automation
Slash commands	Commands you can invoke with `/` in chat

Installed plugin customizations appear alongside your locally defined ones. For example, skills from a plugin show up in **Configure Skills**, and MCP servers appear in the MCP server list. Installing an automation template does not start an automation; review and configure it before turning it on.

Check the plugin's supported clients and components before installing. See Cross-tool compatibility for how portable and client-specific components differ.

> **Caution**
>
> Plugins can include hooks and MCP servers that run code on your machine. Review the plugin contents and publisher before installing, especially for plugins from community marketplaces.

Discover and install plugins

Use the Agent Customizations editor to browse marketplaces and install plugins. You can also install a plugin directly from a Git repository or local folder. Available plugins and actions depend on the selected agent harness and organization policy.

Install a plugin from a marketplace

1. Open the Agent Customizations editor by running **Chat: Open Customizations** from the Command Palette, selecting **Open Customizations** in the Chat view title bar, or selecting **Customizations** in the Agents window sidebar.

2. Browse available plugins from your configured marketplaces. Available sources depend on the selected harness and organization policy.

    - Select the **Browse** tab and search for a plugin.
    - Select the **Plugins** tab and then select **Browse** to find available plugins.

3. Select a plugin to review its publisher, included customizations, and **Plugin README**, when provided.

4. Select **Install** to install a plugin.

   The first time you install a plugin from a new marketplace, VS Code shows a trust prompt. Review the marketplace source before confirming.

Install a plugin from source

You can install a plugin directly from a Git repository or local folder without adding a full marketplace first. For a repository URL, VS Code clones and installs the plugin.

1. In the Agent Customizations editor, select **Plugins**, then select the add (`+`) button next to **User** or **Workspace**, depending on where you want to install the plugin.
2. Enter a Git repository URL (for example, `https://github.com/rwoll/markdown-review`) or a local folder path.

Alternatively, run **Chat: Install Plugin from Source** from the Command Palette.

Plugins installed by GitHub Copilot CLI

VS Code automatically discovers plugins that you install with the GitHub Copilot CLI to enable you to use them also in VS Code. Discovered plugins from `~/.copilot/installed-plugins/` appear on the **Plugins** page of the Agent Customizations editor.

The CLI stores plugins under `~/.copilot/installed-plugins/<marketplace>/<plugin>/`. Plugins installed directly from a Git URL (rather than from a marketplace) live under the `_direct` bucket, for example `~/.copilot/installed-plugins/_direct/github--moda-linter--copilot-plugin/`.

View installed plugins

In the Agent Customizations editor, select **Plugins** to view the plugins available to the selected harness. From this page, you can inspect, enable, disable, or uninstall plugins.

Select an installed plugin to open its details. Review its **Plugin README** and included customizations. Under **Skills**, select a skill to open its instructions.

Verify a plugin after installation

Installing a plugin makes its supported components available; it doesn't guarantee that an agent uses them for every task.

1. In the Agent Customizations editor, select **Plugins** and confirm that the plugin is listed and enabled for the intended harness and workspace.
2. Review the publisher's setup instructions and complete any required configuration. For bundled MCP servers, check server controls and connection requirements.
3. Start a fresh session with the intended harness and workspace. Confirm that the components you need are available. For example, run one of the plugin's skills if it appears in the `/` menu.
4. Try a small task you expect the plugin to help with, then review the agent's actions and result.

If you only need a skill, consider using or adapting an existing skill instead of installing a plugin. To change and maintain a plugin, follow the plugin authoring reference.

Enable or disable plugins

You can enable or disable a plugin for the current profile or a specific workspace:

1. In the Agent Customizations editor, select **Plugins** and open the plugin's details.
2. Use **Enable** or **Disable** to change its state for the current profile, or **Enable (Workspace)** or **Disable (Workspace)** for the current workspace. Use the button's dropdown menu to choose the scope.

The enable/disable state is stored separately from the plugin configuration, so it does not affect shared workspace settings.

Policy-managed plugins are identified as managed by policy, and their toggles are locked.

When a plugin is disabled, its skills, agents, hooks, MCP servers, slash commands, and automation templates are no longer available. For example, skills from a disabled plugin do not appear in **Chat: Configure Skills**, and automation templates do not appear in **Templates from Plugins**. Disabled plugins appear with a dimmed style in the Agent Customizations editor.

Uninstall plugins

In the Agent Customizations editor, select **Plugins**, open the plugin's details, and select **Uninstall**.

Plugins installed from an external source (such as npm, PyPI, or an external Git repository) are removed from disk. Plugins that are inlined in a marketplace repository remain on disk but are no longer active.

Configure plugin marketplaces

Add and manage plugin marketplaces in the Agent Customizations editor or configure their sources in Settings.

Adding a marketplace makes its plugins available to browse. It does not install or enable them.

Manage marketplaces in the Customizations editor (Preview)

1. Open the Agent Customizations editor and select **Browse**.
2. Open the source menu and select **Configure Marketplaces**.
3. Select **Add Marketplace** and enter a GitHub repository, Git repository URL, or local repository URI. Only add marketplaces you trust.
4. Return to **Browse** and select the added marketplace from the source menu to browse its plugins.

To manage an existing marketplace, select it in **Configure Marketplaces**. Available actions depend on the selected harness and include **Browse**, **Refresh Marketplace**, **Open Folder**, and **Remove Marketplace**. Use **Back** to return to the marketplace list.

Organization-managed marketplaces can't be removed, and some harnesses also protect built-in marketplaces. If prompted to **Remove Marketplace and Plugins**, review which plugins will be uninstalled before confirming.

Configure marketplaces in Settings

You can discover plugins from marketplaces such as copilot-plugins and awesome-copilot. To configure additional marketplaces, use `chat.plugins.marketplaces` (Experimental).

Marketplaces are Git repositories that contain plugin definitions. You can reference them in several formats:

- **Shorthand**: `owner/repo` for public GitHub repositories. For example, `anthropics/claude-code`. Append `#<ref>` to select a branch, tag, or full 40-character commit SHA.
- **HTTPS git remote**: a full URL ending in `.git`. For example, `https://github.com/anthropics/claude-code.git`.
- **SCP-style git remote**: SSH-style references. For example, `git@github.com:anthropics/claude-code.git`.
- **file URI**: a `file:///` path to a marketplace repository already cloned on disk.

For reproducible installations, use a full 40-character commit SHA, such as `owner/repo#<commit-sha>`. VS Code installs plugins from that exact revision. Branch and tag references use the revision they resolve to when the marketplace is fetched.

Private repositories are also supported. Sign in to GitHub and make sure you have read access to the marketplace repository. Organization membership alone does not grant repository access. If a public lookup

fails, VS Code falls back to cloning the repository directly.

When a marketplace index declares a plugin at a relative path, VS Code verifies the repository, resolved revision, and path before installation. Sources with a missing revision, an invalid path, or an ambiguous path match are rejected.

Marketplace plugins can also reference external package sources such as npm or PyPI packages. For the full marketplace plugin schema, see the Claude Code plugin marketplace documentation.

```
// settings.json
"chat.plugins.marketplaces": [
    "anthropics/claude-code"
]
```

> **Note**
>
> Enterprise admins can centrally control which plugins and marketplaces are available to developers. For more information, see Manage agent plugins and marketplaces.

Use local plugins

To install a plugin from a local folder, use Install a plugin from source in the Agent Customizations editor.

If you manually clone or download a plugin and need to register its directory without installing it, use `chat.pluginLocations` (Experimental). This setting maps local plugin directory paths to an enabled or disabled state. Set the value to `true` to enable the plugin, or `false` to keep it registered but disabled.

```
// settings.json
"chat.pluginLocations": {
    "/path/to/my-plugin": true,
    "/path/to/another-plugin": false
}
```

Update plugins

In the Agent Customizations editor, select **Plugins**, then select **Check for and Apply Updates** (the refresh icon) next to an installed-plugin group. Alternatively, run **Chat: Update Plugins** from the Command Palette.

Updating pulls down changes from cloned marketplace repositories and checks for new versions of externally sourced plugins.

Plugins can also update automatically when `extensions.autoUpdate` is enabled. Plugins sourced from npm or PyPI never update automatically. Use the update action in the Agent Customizations editor and review the confirmation prompt before the install command runs.

Workspace plugin recommendations

Projects can recommend plugins for team members by configuring plugin settings in the workspace settings (`.claude/settings.json` or `.github/copilot/settings.json`).

In the Local harness, VS Code shows a notification the first time a chat message is sent. To browse and install plugins from the project's marketplaces, open the Agent Customizations editor and select **Browse**, or select **Plugins** > **Browse Marketplace** if the unified Browse experience is unavailable.

In the Copilot harness, `.github/copilot/settings.json` supplies repository plugin and marketplace defaults. You can turn off optional plugins in **Plugins** in the Agent Customizations editor without changing the shared defaults. Enterprise-managed settings take precedence over repository defaults and your choices.

Specify the following fields in the settings file to configure workspace plugin recommendations:

- **`extraKnownMarketplaces`**: registers additional marketplaces for the project. These marketplaces appear among the available sources in the Agent Customizations editor.
- **`enabledPlugins`**: lists plugins that should be enabled by default.

```
{
  "extraKnownMarketplaces": {
    "company-tools": {
      "source": {
        "source": "github",
        "repo": "your-org/plugin-marketplace"
      }
    }
  },
  "enabledPlugins": {
    "code-formatter@company-tools": true
  }
}
```

The marketplace key, `company-tools` in this example, must match the `name` in the marketplace's `marketplace.json`. Plugin identifiers use `<plugin>@<marketplace>`, not the marketplace repository name. Keep marketplace names stable so shared settings continue to refer to the same plugins.

Manage plugins with slash commands

As an alternative to the Agent Customizations editor, you can manage plugins with slash commands in a Copilot harness session. Enter `/plugin` in the chat input to view completions for plugin and marketplace

operations.

> **Note**
>
> The `/plugin` commands and the Agent Customizations editor currently use separate plugin inventories. Manage a plugin from the interface where you installed it.

Expand table

Command	Purpose
`/plugin list`	List installed plugins and their status.
`/plugin install <source>`	Install a plugin from a marketplace or Git repository.
`/plugin update <plugin>`	Update an installed plugin.
`/plugin enable <plugin>`	Enable an installed marketplace plugin.
`/plugin disable <plugin>`	Disable an installed marketplace plugin.
`/plugin uninstall <plugin>`	Remove an installed plugin.
`/plugin marketplace list`	List included and registered marketplaces.

`/plugin marketplace browse <name>`	Browse the plugins in a marketplace.
`/plugin marketplace add <source>`	Add a marketplace from a Git repository.
`/plugin marketplace update [name]`	Update one marketplace or all registered marketplaces.
`/plugin marketplace remove <name> [--force]`	Remove a registered marketplace. Use `--force` if plugins from the marketplace are installed.

Marketplace plugins use the qualified identity `<plugin>@<marketplace>`. Use this identity when plugins in multiple marketplaces have the same name. Command completion suggests valid marketplace and plugin identities for each operation.

Plugins installed directly from a Git repository can be updated or uninstalled, but they can't be enabled or disabled. To stop using a directly installed plugin, uninstall it. Included and centrally managed marketplaces can't be removed.

Cross-tool compatibility

Agent Plugins is an open standard for packaging skills and MCP servers across compatible clients, including GitHub Copilot in VS Code, GitHub Copilot CLI, and the GitHub Copilot app. Other components, such as custom agents, slash commands, rules, hooks, and automation templates, can be client-specific.

VS Code also supports existing Copilot, Claude, and legacy OpenPlugin formats. Support for a package format doesn't guarantee that every component works in every harness or client. Check the plugin's requirements. For file layouts and client-specific namespaces, see the plugin authoring compatibility reference.

Troubleshooting

Plugin does not appear after installation

- Confirm that agent plugins are enabled: check that `chat.plugins.enabled` is set to `true`.
- Verify the plugin's `name` field follows the naming rules for its format. Agent Plugins 1.0 names use lowercase letters, numbers, hyphens, and periods. Legacy Copilot plugin names use lowercase letters, numbers, and hyphens. Slashes and colons aren't supported.
- Check that `plugin.json` is in a recognized location.

Skills from a plugin do not load

- Open the `SKILL.md` file and check the `name` field in the YAML frontmatter. The name must be plain kebab-case without namespace prefixes (for example, `test-runner`, not `myorg/test-runner`). Invalid names cause the skill to be silently skipped.
- Make sure the skill directory name matches the `name` field in the `SKILL.md` frontmatter.

Plugin version does not update

- In the Agent Customizations editor, select **Plugins**, then select **Check for and Apply Updates** (the refresh icon).
- If you maintain the plugin, review its versioning and update requirements.

Plugin installation fails after retry

Retry the installation. VS Code validates marketplace repository caches and replaces incomplete or invalid caches automatically.

If installation still fails, review the error for an invalid repository, revision, or plugin path. If the source is valid and retry continues to fail, run **Help: Report Issue**. Don't delete or modify the internal plugin cache manually.

Author a plugin

Use the agent plugin authoring reference to create a package or inspect its configuration:

- **Package structure:** Manifest fields, supported formats, and plugin environment variables.
- **Automation templates:** Package prompts and schedules.
- **MCP servers:** Server configuration, paths, and interaction with other servers.
- **Hooks:** Hook configuration, paths, and events.

Related resources

- Finding and installing plugins for GitHub Copilot CLI
- GitHub Copilot CLI plugin reference

Was this page helpful?

Yes No

Still need help?

Ask the community

Request features

Report issues

Help us improve

Edit this page

10/28/2026